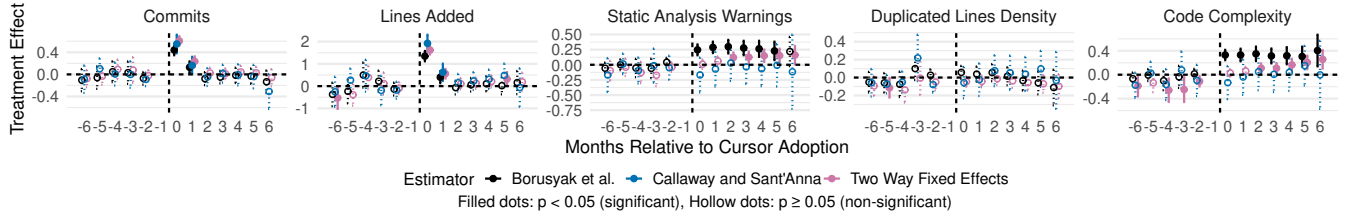


Table 6: The estimated average treatment effects on treated (i.e., ATT) post Cursor adoption from different DiD estimators. Similar to Table 2, all outcome variables are log-transformed, and percentage changes are provided for reference.

Outcome	Borusyak et al. [29]			Two-Way Fixed Effects			Callaway and Sant'Anna [31]		
	Estimate (Std. Err.)	% Change		Estimate (Std. Err.)	% Change		Estimate (Std. Err.)	% Change	
Commits	0.0260 (0.0429)	+2.63% ($\pm 4.40\%$)		0.1641*** (0.0386)	+17.83% ($\pm 4.55\%$)		-0.0073 (0.0695)	-0.73% ($\pm 6.90\%$)	
Lines Added	0.2514* (0.1063)	+28.58% ($\pm 13.7\%$)		0.6029*** (0.0876)	+82.74% ($\pm 16.0\%$)		0.4292** (0.1536)	+53.60% ($\pm 23.6\%$)	
Static Analysis Warnings	0.2644*** (0.0511)	+30.26% ($\pm 6.66\%$)		0.1696*** (0.0415)	+18.48% ($\pm 4.92\%$)		-0.1108 (0.1254)	-10.49% ($\pm 11.2\%$)	
Duplicated Lines Density	0.0679 (0.0448)	+7.03% ($\pm 4.79\%$)		0.0160 (0.0390)	+1.61% ($\pm 3.96\%$)		-0.0034 (0.0785)	-0.34% ($\pm 7.82\%$)	
Code Complexity	0.3481*** (0.0538)	+41.64% ($\pm 7.62\%$)		0.2314*** (0.0446)	+26.04% ($\pm 5.62\%$)		-0.0387 (0.1136)	-3.80% ($\pm 10.9\%$)	

Note: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$ **Figure 6: The estimated horizon-average treatment effects (ATT_h) from three DiD estimators: Borusyak et al. [29], Callaway and Sant'Anna [31], and two-way fixed effects. The estimates on development velocity outcome are highly consistent and robust across all three estimators, but the estimates on software quality outcomes vary significantly.**

+18.48%) and code complexity (+41.64% and +26.04%), while the Callaway and Sant'Anna [31] estimator yields negative but statistically insignificant estimates for these same outcomes (-10.49% and -3.80%). All three estimators find no significant effects on duplicated lines density after Cursor adoption.

The divergence between Callaway and Sant'Anna [31] and the other estimators warrants careful interpretation, which may be attributed to several methodological differences. First, as discussed in Section B.2, the Callaway and Sant'Anna [31] estimator estimates treatment effects separately within each cohort before aggregating. In our setting, many adoption cohorts contain fewer than 100 repositories (Figure 2), meaning cohort-specific estimates rely on limited sample sizes. Combined with the inherent noisiness of code quality metrics (e.g., measurement errors in static analysis tools and high variation in code characteristics), this small-cohort structure likely reduces statistical power to detect genuine effects. Second, the Callaway and Sant'Anna [31] estimator conditions only on pre-treatment covariates $Z_{i,g-1}$ (Equation 13), whereas both TWFE and Borusyak et al. [29] adjust for time-varying covariates Z_{it} . If time-varying confounders influence code quality outcomes beyond what pre-treatment characteristics capture, this difference in covariate adjustment could contribute to the divergent estimates. Still, there is no clear consensus on when time-varying covariates help or hurt causal inference in staggered DiD settings.

We report the Borusyak et al. [29] estimator in the main paper for three reasons: (1) It avoids the biases of TWFE while maintaining statistical power by pooling untreated observations; (2) it passes pre-trend tests across all outcomes; and (3) the sustained temporal patterns in Figure 6 support the causal interpretation. We acknowledge that the lack of large cohorts and clear theoretical guidance on the incorporation of time-varying covariates in DiD

limits our ability to definitively resolve estimator disagreements for code quality outcomes; the findings around code quality outcomes should be interpreted with appropriate caution.

C Full Robustness Check Results

Following discussions in Section 4.3, we present summary statistics in all alternative dataset settings in Table 7 and additional robustness check results in Figure 7. Apart from the settings already discussed in Section 4.3, we also conduct experiments across different programming language groups to determine whether our findings are driven by repositories in particular programming languages or are consistent across programming languages. The results in Figure 7, Row 1 show that our main causal findings are qualitatively consistent across all the major programming languages in our dataset (JavaScript/TypeScript, Python, Go). Comparing the differences, we observe that the velocity gain is most transient in JavaScript/TypeScript repositories but less so in Python and Go repositories, while the effect on code quality in Go repositories is strongest. However, it is essential to note that this heterogeneity should not be attributed to the programming languages themselves in our setting. Our intuition is that some form of selection bias (e.g., Go repositories may be larger and more sustained) is driving the heterogeneity we observe here. Also, our dataset does not sufficiently cover several major programming languages, such as Java, C/C++, and Rust, in which the outcomes of AI coding tool adoption might differ substantially from those in Python and JavaScript projects (the latter have more training data and better LLM performance as of now). Therefore, we believe that future research is necessary to explore the impact of AI coding tools on other programming languages and the mechanisms underlying these heterogeneities.